COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

09

Co-design reference case study

Functional models, analytical estimates and native host results

PACKAGE 09 / VOLUME 08 / SEPTEMBER 28, 2026
Modules 36–40 · Gate H · Expanded teaching edition 1.0
Study, implement, test and review. Models are not physical measurements.

Contents

Modeled alternatives 3
Host measurements 4
What the learner still must do 4
How this example was generated 5
Interpreting the comparison 5
Files to inspect 5
Sensitivity of the teaching model 6

The reference case is a bounded fixed-point affine transform. It is a teaching example, not a production engine workload.

Modeled alternatives

36 cases compare 1-, 2- and 4-lane alternatives at six sizes with normal and backpressured output. Every value was checked against the integer oracle. Timing assumes a 100 MHz clock; the clock and resource counts are NOT synthesis results.

The analytical end-to-end model includes 20 microseconds setup, 5 microseconds completion, 100 MB/s host payload transfer and 400 MB/s internal payload service. Units are decimal. Zero-size calls are skipped. These constants are educational assumptions, not hardware specifications.

N	Lanes	Adverse	Modeled kernel us	Modeled end-to-end us
1	1	False	0.040	25.080
1	1	True	0.040	25.080
1	2	False	0.040	25.080
1	2	True	0.040	25.080
1	4	False	0.040	25.080
1	4	True	0.040	25.080
64	1	False	0.670	28.230
64	1	True	1.310	28.870
64	2	False	0.640	28.200
64	2	True	0.670	28.230
64	4	False	0.640	28.200
64	4	True	0.640	28.200
65536	1	False	655.390	3301.830
65536	1	True	1310.750	3957.190
65536	2	False	655.360	3301.800
65536	2	True	655.390	3301.830
65536	4	False	655.360	3301.800
65536	4	True	655.360	3301.800

Four lanes do not automatically beat one. At the declared internal bandwidth, 4 bytes per item imposes 10 ns per item, already matching the one-lane asymptotic compute interval at 100 MHz. More lanes mainly shorten startup/drain effects or mitigate selected stalls; changing memory bandwidth can change that result.

Host measurements

These are real timings of the provided C++ programs on the build host, not FPGA/GPU timings and not a measurement of the learner's machine. The contiguous loop is optimized by the compiler; the blocked loop is a second software candidate, not a promised improvement. No Python-to-hardware speedup claim is made.

N	Variant	Median ns	p95 ns	Samples
0	contiguous	30.0	31	30
0	blocked	30.0	31	30
1	contiguous	32.0	33	30
1	blocked	33.0	34	30
7	contiguous	46.0	50	30
7	blocked	53.0	54	30
64	contiguous	190.0	194	30
64	blocked	204.0	209	30
1024	contiguous	2629.5	4062	30
1024	blocked	2703.5	5869	30
65536	contiguous	166261.0	284377	30
65536	blocked	168627.5	236659	30

What the learner still must do

Choose and measure an actual engine workload; set application-specific quality and latency tolerances; reproduce a competent baseline; adapt at least two architecture alternatives; sweep setup, transfer and internal bandwidth; retain small, large and adverse cases; and defend an independently reproduced conclusion. A software-only winning design is a valid outcome.

The supplied pipeline is an educational behavioral register model. It is not a tested board, synthesizable RTL, an ASIC estimate, an electrical design approval or evidence of console SDK access.

How this example was generated

Run `python -m capstone.run_capstone --native --output my_results/native` from the package root when a C++17 compiler is available. Omit --native for the Python-only model. The provided result directory captures this edition's actual run; use a new output directory for your own results.

The fixed-point oracle, pipeline output and native checksum are compared before results are accepted. Six sizes are 0, 1, 7, 64, 1,024 and 65,536. Three lane counts are 1, 2 and 4. Normal readiness is always true; adverse readiness repeats true/false/false/true. Pipeline latency is declared as four cycles; global stall and endpoint conventions are documented in the source. Arithmetic is evaluated at acceptance and delayed through registers, so this is a behavioral packet pipeline, not synthesized arithmetic timing.

The native source uses C++17 at -O3, five warmup invocations per candidate and 30 retained samples per candidate/size, alternating candidate order. It checks each output against an independent exact-double oracle for this bounded binary arithmetic. Allocation, oracle validation and checksum are outside the timed function interval. A separate Python calculation verifies each recorded native checksum. Raw CSV files retain warmup and sample rows with phases.

No CPU pinning, frequency control, thermal measurement or device-level power instrumentation was performed. Nanosecond output units do not establish timer accuracy. Empty and very small batches can be timer-dominated. This host environment is not claimed to be representative of a particular gaming PC, handheld or console. Consult environment.json for the actual interpreter, compiler and flags.

Interpreting the comparison

At N=65,536 with normal readiness, the four-lane modeled end-to-end result is about 3.302 ms under the stated assumptions. Host baseline measurements in the table are function-only and much smaller in this run. This suggests the proposed transfer/setup model is unattractive for this bounded operation, but it is not a measured hardware comparison or a complete engine conclusion. The assumed device transfer path is a different execution boundary from the native arithmetic-only timing; a learner must reconcile all preparation and result-consumption costs before a product decision.

The useful lesson is that specialization can lose despite attractive compute parallelism. Internal bandwidth limits lane scaling and host transfers dominate end-to-end time. Raising internal bandwidth alone does not remove the host-transfer bottleneck. Sweep both resources and setup independently, then connect the operation to a measured frame fraction before claiming application benefit.

Files to inspect

`model_results.json` contains every modeled case and assumption. `example_pipeline_trace.json` shows the sampled packet-flow timeline and exact outputs. `native_n*.csv` retain raw host timing records. `native_summary.json` contains native summaries. `environment.json` records method and limits. `evidence_ledger.json` links declared claims to hashed artifacts. `native_build.log` preserves compiler output; an empty log indicates no compiler messages, not a missing attempt.

The standard-library auditor checks ledger structure, allowed relative paths, file existence and SHA-256 matches. It does not decide whether the scientific interpretation is sound. The portfolio review must still challenge workload relevance, baseline fairness and every modeled-to-measured comparison.

Sensitivity of the teaching model

972 analytical combinations reuse the 36 validated functional traces. Setup, host payload bandwidth and internal payload bandwidth each take three declared values. These are hypothetical assumptions, not measured device properties.

The table holds setup at 20 us, completion at 5 us and selects N=65,536, four lanes, normal readiness. Only the two bandwidth assumptions change.

Host MB/s	Internal MB/s	Conditional end-to-end us
25	100	13132.200
25	400	11166.120
25	1600	10674.630
100	100	5267.880
100	400	3301.800
100	1600	2810.310
400	100	3301.800
400	400	1335.720
400	1600	844.230

Higher internal bandwidth alone does not remove host transfers. At each bandwidth pair, setup values 5 and 80 us shift a nonempty result by −15 and +60 us relative to this table. An engine decision still needs measured workload frequency and an application critical-path model.

Reproduce in a new folder with: `python -m capstone.sensitivity --input my_results/model_only/model_results.json --output my_results/model_only`. Changing compute architecture or clock requires a corresponding justified model, not only editing these costs.